



Validation: Findings and Optimizations

This document summarizes security findings, fixes, and performance optimizations applied to the inference validation flow (including revalidation and invalidation throttling).

Important: Two-phase patch application

This validation patch should be applied in two phases.

- **Phase 1 (initial rollout):** Deploy with the designated-validator check (`ShouldValidate`) **skipped when the participant's random seed is not found**. In `msg_server_validation.go`, when `GetParticipantEpochSeed` returns `!found`, the code currently sets `skipTheShouldValidateCheck = true` so validation/revalidation is still accepted. This is required because **seeds are only stored when the next epoch starts**; right after the patch, existing epochs may have no seeds in storage, and rejecting all validations would break the flow.
- **Phase 2 (after seeds are populated):** Once the chain has run long enough that seeds are stored for the epochs that need validation (e.g. after the next epoch transition), **remove the skip** and enforce the designated-validator check: if the seed is not found, return an error (e.g. `types.ErrRandomSeedNotFound`) instead of allowing the message. See the TODO in `msg_server_validation.go` around the `skipTheShouldValidateCheck` block (lines 127–135).

Do not remove the skip until seeds are reliably present for the epochs in use, or validators will be incorrectly rejected.

1. Caching: EpochData and Seeds

Implemented

- `EpochGroupData` cache

A hot cache for `EpochGroupData` keyed by `(epochIndex, modelId)`:

- **Scope:** Current and previous effective epoch only. Other epochs are not cached.
- **Lifecycle:** Cache is refreshed when the effective epoch changes (`SetEffectiveEpochIndex`). The old “previous” epoch is dropped; the old “current” becomes the new “previous.”
- **Write-through:** Updates for current/previous epoch go to cache; current-epoch cache is flushed to storage in `EndBlock` (`FlushCurrentEpochGroupCache`).
- **Relevance:** Addresses the issue where repeated reads of epoch group data for the same epoch/model were hitting storage on every validation path.

- **RandomSeed cache**

We need a seed as we check if participant is eligible to validate the inference.

A warm cache for participant seeds per epoch:

- **Scope:** Current effective epoch only.
- **Lifecycle:** Cleared and re-initialized when the effective epoch changes (`refreshRandomSeedCache`).
- **Usage:** `GetRandomSeed` / `GetParticipantEpochSeed` use the cache so that `calculations.ShouldValidate` can be run without extra storage reads for seeds in the current epoch.

- **Normalized participants tree cache (block hash/height bound)**

In-memory cache keyed by **block hash**: each entry is a BTree mapping cumulative normalized weight → participant address, used for weighted sampling (e.g. revalidation).

One entry is written per committed block when the Precommitter runs

(`SetNormalizedParticipantsForCommittedBlock`), using the block’s height and hash.

Lookup is via `GetNormalizedWeightedParticipants(blockHash)`. **Eviction:** Entries older than 300 blocks are dropped at block start: in `PrepareForBlock` we call

`normalizedWeightedParticipants.ClearByHeight(currentBlockHeight - NormalizedParticipant`
so only the last 300 blocks are kept. See

`x/inference/keeper/normalized_weighted_participants_cache.go` (constant `NormalizedParticipantsCacheBlocks = 300`, `Add`, `ClearByHeight`, `Get`) and the eviction call in `x/inference/keeper/revalidation_init_hook.go` (`PrepareForBlock`).

Tx-bound draft for EpochGroupData (isolation during transaction)

Cosmos SDK can revert a transaction on error (e.g. failed check, panic). Any in-memory

cache updated during that tx would then reflect changes that never committed. To keep the shared EpochGroupData cache isolated per transaction we use a **tx-scoped draft** that is only merged into the real cache when the tx succeeds.

Implementation:

- **Context key:** A private type `ctxKeyEpochGroupDraft` is used as the context key. The value is a pointer to `map[epochGroupCacheKey]types.EpochGroupData` (the draft map).
- **WithEpochGroupDraft(ctx):** Allocates a new draft map and attaches it to the context with `context.WithValue(ctx, ctxKeyEpochGroupDraft{}, &draft)`. Returns a new context; the draft is not shared across txs (works with parallel execution).
- **getEpochGroupDraftFromContext(ctx):** Returns the draft map from the context, or `nil` if no draft is bound.

Flow:

1. Tx start (AnteHandler)

`EpochGroupDraftDecorator.AnteHandle` runs before the tx is executed. It takes the SDK context's underlying `context.Context`, calls `WithEpochGroupDraft(base)`, and re-attaches the new context to the SDK context via `ctx.WithContext(newBase)`. Every subsequent keeper call in this tx sees the same context and thus the same draft.

2. During tx execution

- **GetEpochGroupData:** If the context has a draft and the requested `(epochIndex, modelId)` is for the current or previous effective epoch, the keeper checks the **draft first**. If the key is in the draft, it returns that value (uncommitted writes visible within the tx). Otherwise it falls back to the real cache and then the store.
- **SetEpochGroupData:** If the context has a draft and the epoch is current/previous, the write goes **only to the draft** (no update to the shared cache or store for current/previous). Other epochs still write through to the store.
- **RemoveEpochGroupData:** If there is a draft, the key is removed from the draft so it is not re-applied on commit.

3. Tx success (PostHandler)

The app registers a PostHandler that runs after each tx. Only when the tx is **successful** it calls `keeper.CommitEpochGroupDraftFromContext(ctx)`. That reads the draft from the context and merges it into the keeper's real `epochGroupCache` (for current/previous epoch keys); non-current epochs are written through to the store. After merge, the draft is

no longer used.

4. Tx failure

If the tx fails or panics, the PostHandler does not run (or runs with `success == false`), so `CommitEpochGroupDraftFromContext` is never called. The draft is discarded when the context is released; the shared cache and store are unchanged.

5. Persistence

The real cache is still only in-memory. Current-epoch entries are flushed to the store in **EndBlock** via `FlushCurrentEpochGroupCache`. So: draft → (on success) real cache → (in EndBlock) store.

Result: EpochGroupData updates for the current/previous epoch are isolated to the tx until it succeeds. Reverted txs never pollute the shared cache or the store.

Revalidation events hook and block hash

Revalidation events are `inference_validation` events emitted by the validation msg server with attribute `needs_revalidation=true`. The app hooks into them so that after a block is finalized we can run revalidation logic (e.g. weighted sampling) using the **block hash** of the block where the event was included.

How we hook to revalidation events:

1. Event emission

In `msg_server_validation.go`, after processing a validation (or revalidation) message, the handler emits an event:

```
Event("inference_validation",
    Attribute("inference_id", ...),
    Attribute("validator", ...),
    Attribute("needs_revalidation", "true"/"false"),
    Attribute("passed", ...))
```

2. Collection (PostHandler)

The app registers a **PostHandler** that runs after each **successful** tx. It reads `ctx.EventManager().Events()`, and for every event of type `inference_validation` with `needs_revalidation=true` (and non-empty `inference_id` and `validator`) it

appends a `RevalidationEventInfo` to a **block-scoped collector**, keyed by **current block height** (`ctx.BlockHeight()`). So during block N we accumulate one list of revalidation events per height N.

3. Only committed execution

At **block start** (`BeginBlock`), `PrepareForBlock(currentBlockHeight)` is called. If the collector supports `ClearEventsForHeight`, we call `ClearEventsForHeight(currentBlockHeight)`. That clears the buffer for the **current** block height. So if the same block is re-executed (e.g. consensus round failed), we do not accumulate events from the failed run; only events from the execution that eventually commits remain when we process them.

4. Processing and block hash (Precommiter)

The app wires a **Precommiter** hook via `RevalidationCommitOption(keeper)`. When the block is **committed**, the SDK runs this hook. At that moment the block has been finalized, so:

- `height := ctx.BlockHeight()` is the height of the block we just committed.
- `hash := ctx.HeaderInfo().Hash` is that block's **block hash** (from the header of the block being committed).

The Precommiter then:

- Calls

```
keeper.ProcessPendingRevalidationEvents(context.Background(), height, hash)
```

: the keeper uses

```
BlockRevalidationEventsProvider.GetInferenceValidationRevalidationEvents(ctx, height)
```

to get all revalidation events for that height (and removes them from the collector).

For each event it calls

```
OnInferenceValidationNeedsRevalidation(ctx, inferenceId, validator, blockHeight, blockHash)
```

— so the block hash passed in is exactly the hash of the block where the event was included.

- Calls `keeper.SetNormalizedParticipantsForCommittedBlock(ctx, height, hash)` to build and cache the normalized weighted participants for this block, keyed by `blockHash`, for use in weighted sampling (e.g. revalidation).

Summary: Revalidation events are collected per successful tx in the PostHandler (by block height). They are processed only once the block is committed, in the Precommiter, where we have the final **block hash** from `ctx.HeaderInfo().Hash`. That block hash is passed to `OnInferenceValidationNeedsRevalidation` and used to key the normalized-participants

cache so downstream logic can use the correct block-bound randomness.

Deterministic revalidation participants (normalized tree cache)

We use the normalized participants tree cache so that **which participants should vote on a given revalidation is deterministic and identical on every node**. It is fully determined by:

- **Inference id** — the inference that needs revalidation (from the event).
- **Block hash** — the hash of the block where the revalidation event was emitted (the committed block we get in the Precommiter).

How it works:

1. **Cache key:** The normalized tree is stored per **block hash**. For each committed block, the Precommiter calls `SetNormalizedParticipantsForCommittedBlock(ctx, height, hash)` so the tree for that block (cumulative normalized weight → participant address) is available via `GetNormalizedWeightedParticipants(blockHash)`.
2. **Deterministic seed:** From `(blockHash, inferenceId)` we derive a single deterministic seed using the same method as elsewhere:
`calculations.SeedFromBytes(blockHash || inferenceId)` (SHA-256, first 8 bytes as int64). No other inputs (time, node, etc.) are used.
3. **Deterministic PRNG:** We instantiate `math/rand.New(rand.NewSource(seed))` with that seed. The sequence of random numbers (e.g. `Float64()`) is therefore fixed for that `(blockHash, inferenceId)` pair.
4. **Weighted sampling:** We draw random values in $[0, 1)$ from that PRNG and, for each value, find the participant whose cumulative normalized weight segment contains it (by ascending the tree and taking the first key $\geq$ the value). We collect up to `NormalizedParticipantsSampleSize` **unique** participants (re-drawing when we hit a duplicate, until we have enough or the tree is exhausted).

Result: For a given revalidation event (inference_id + block hash where the event was emitted), every node computes the **same** set of participants who should vote. There is no per-node or per-call randomness; the same `(blockHash, inferenceId)` always yields the same ordered set of participants, weighted by their confirmation weight in that block's epoch group. Implementation: `SampleNormalizedParticipantsForInference(blockHash, inferenceId)` in

x/inference/keeper/revalidation_init_hook.go .

Performance characteristics:

- **Seed derivation:** $O(\text{len}(\text{blockHash}) + \text{len}(\text{inferenceId}))$ for SHA-256; one hash per $(\text{blockHash}, \text{inferenceId})$.
 - **Sampling:** Up to `NormalizedParticipantsSampleSize` (10) draws, each a `Float64()` plus a lower-bound lookup for the first key $\geq r$. We use `Ascend(r, ...)`, which seeks to the first key $\geq r$ in $O(\log P)$ and yields that element; we take only the first callback result. So per-draw cost is $O(\log P)$ where P = participants in tree, i.e. $10 \cdot \log(P)$ total for the sampling loop.
 - **No storage reads:** Uses only the in-memory normalized tree (keyed by block hash) and in-memory PRNG; no KV or collections access.
 - **Cached vote list:** The resulting list of selected participants is cached in an in-memory map keyed by $(\text{blockHeight}, \text{inferenceId})$, populated when revalidation events are processed in the Precommiter (after the normalized tree for that block is set). Entries older than `NormalizedParticipantsCacheBlocks` (300) blocks are cleared in `PrepareForBlock`, in the same way as the normalized participants cache. Eligibility checks via `IsParticipantEligibleToVoteOnRevalidation(blockHeight, inferenceId, participantAddress)` are $O(1)$ map lookup plus $O(k)$ slice membership where $k \leq 10$.
-

Capped revalidation vote weights (20% with redistribution)

Revalidation vote weights are **capped** so that no single participant can dominate the outcome. We apply a **hard cap of 20%** of the total eligible weight per participant. The cap is chosen so that **at least 3 validators** are required to reach a majority (50%): with each vote at most 20%, three votes sum to at most 60%, so reaching 50% still requires multiple participants.

How it works:

- When revalidation events are processed (`ProcessPendingRevalidationEvents`), we build the set of selected participants (invalidator + sampled participants) and their **raw** weights from the normalized block weights.

- We compute **total eligible weight** as the sum of those participants' raw weights.
- **Cap limit** = 20% of total eligible weight. Each participant's vote weight is then **min(raw weight, cap limit)**. No redistribution of "excess" weight is applied; the effective total used for threshold is the sum of these capped weights.
- Capped weights are stored in the **revalidation vote participants cache** keyed by `(inferenceId, participant address)`, so when a participant submits a revalidation vote we use their **capped** weight (via `GetRevalidationVoteWeight`) rather than raw confirmation weight. The cache is evicted after `NormalizedParticipantsCacheBlocks` (300) blocks, together with the normalized participants tree.

Implementation: `x/inference/keeper/revalidation_init_hook.go` (cap and cache population), `revalidation_vote_participants_cache.go` (storage and `GetWeight`), and `msg_server_validation.go` (use capped weight when adding a revalidation vote).

Why we don't use x/group for revalidation voting

Revalidation uses **small, randomly sampled groups** of participants (e.g. up to `NormalizedParticipantsSampleSize` per inference) chosen deterministically from the normalized tree. We **do not use Cosmos SDK x/group** for this voting because:

- **Overhead of x/group:** Creating a group in x/group and running its proposal/voting flow involves significant on-chain work (group creation, proposal submission, vote tallying, execution). Doing that for every revalidation would mean a new group (or proposals on an existing group) per inference that needs revalidation.
- **Small, ephemeral sets:** Our revalidation voters are a small, per-inference random subset that exists only for a short window (e.g. 300 blocks). Modeling each as an x/group group would be heavy and unnecessary.
- **Keeper/cache path:** Instead, we use the **keeper path**: votes are either stored in keeper collections (`InferenceRevalidations`) or only in an in-memory cache, with a simple 50% weight threshold. This matches the small random groups and avoids x/group creation and voting overhead.

So revalidation voting is implemented entirely in the inference module (keeper + optional ephemeral cache), not via x/group.

Revalidation vote storage: `storeRevalidationVotes` (keeper vs cache only)

The keeper has a flag `storeRevalidationVotes` that controls where revalidation votes are stored:

- `storeRevalidationVotes == false` (default)

Votes are kept **only in the in-memory ephemeral cache**

(`ephemeralRevalidationVoteCache`). No revalidation votes are written to chain storage.

The cache is evicted after `NormalizedParticipantsCacheBlocks` (300) blocks in

`PrepareForBlock` . This avoids extra storage writes and keeps revalidation lightweight; suitable when the chain does not need to persist vote history.

- `storeRevalidationVotes == true`

Votes are stored in **keeper collections** (`InferenceRevalidations` ,

`InferenceRevalidationTotalEligibleWeight`). The same 50% threshold logic applies; totals are computed from stored votes. Use this when vote history or auditability is required.

The app can set the mode after keeper construction via

`keeper.SetStoreRevalidationVotes(true)` if persistence is desired; otherwise the default is cache-only.

Planned extension

- Any further extensions to revalidation vote weights or audit structures can build on the above (capped weights in cache and optional persistent storage).
-

2. Moving `addInferenceToEpochGroupValidations` After Checks

Problem

Previously, `addInferenceToEpochGroupValidations` was invoked **before** the main validation checks in `Validation`. That caused possible load:

1. **Spam / griefing:** A non-participant (or any account) could send a `MsgValidation` for an inference.
2. **Early storage write:** `addInferenceToEpochGroupValidations` created the inference validation record in chain storage (relatively heavy operation).

Fix

`addInferenceToEpochGroupValidations` is now called **only after** all of the following have passed:

- Participant and inference existence and status checks
- Executor and model checks
- Epoch group and group data checks
- **Designated-validator check** (see Section 3)
- For non-revalidation: only then is the inference added to epoch group validations

So storage is only updated when the sender is allowed to validate and other preconditions hold. Even if data wasn't stored as errors are thrown, this small tweak reduced possible load on the system.

3. Designated-Validator Check (Eligibility to Validate)

Problem

There was no check that the sender of a validation or revalidation message was the

designated validator for that inference (as determined by the PoC logic). That enabled:

1. **Free-riding:** Any participant could send validation messages for inferences they were not assigned to validate, get them accepted, and share rewards without doing the work.
2. **Sybil / strike evasion:** An attacker with multiple addresses could have one address execute the inference and others “validate” it. Invalid inferences could be “validated” by colluding addresses and the executor could avoid being struck.
3. **Revalidation DoS:** Any participant could trigger the revalidation procedure. Because revalidation is computationally costly, this could be used as a DoS vector.

Fix

Before performing validation or revalidation, the handler now:

1. Loads **inference validation details** (epoch, inference id, total power, executor power, etc.) and the **participant’s seed** for that epoch (using the seed cache when possible).
2. Calls

`calculations.ShouldValidate(participantSeed, inferenceDetails, totalWeight, validatorP`
to determine if this participant is the designated validator for this inference.

3. If `ShouldValidate` returns false, the message is rejected with `types.ErrNotDesignatedValidator` and no state change is applied (no validation record, no revalidation).

So only the participant selected by the deterministic PoC logic can validate or trigger revalidation for a given inference.

Proposal: Strike for Ineligible Validation/Revalidation

Recommendation: Treat submission of validation or revalidation messages by a participant who is **not** the designated validator as misbehavior and **strike** that participant (e.g. reputation penalty or slashing, according to chain policy). This would:

- Discourage probing or abuse once the eligibility check is in place.
- Align incentives with the design (only the designated validator should ever send such messages).

Implementation would hook into the existing strike/slashing machinery and trigger when `ShouldValidate` is false for the sender of a `MsgValidation` (including revalidation).

4. Tiny Performance Optimizations

- **EpochGroupData:** Repeated reads for the same `(epochIndex, modelId)` in the validation path use the EpochGroupData cache instead of storage.
 - **Seeds:** Participant seed for the current epoch is served from the RandomSeed cache when available, reducing storage reads in `GetParticipantEpochSeed` used by `ShouldValidate`.
 - **Invalidations:** Existing TODOs note possible caching for `CountInvalidations` and for `GetSummaryByModelAndTime` to further reduce work in the invalidation-throttling path; these can be addressed in follow-up changes.
-

Summary

Area	Change	Purpose
Caches	EpochGroupData + RandomSeed caches (current/previous epoch)	Fewer storage reads; EpochData cache tied to the related issue.
Ordering	<code>addInferenceToEpochGroupValidations</code> after all checks	Prevents spam from creating validation records and blocking legitimate validators.
Eligibility	<code>ShouldValidate</code> before accepting validation/revalidation	Stops free-riding, Sybil/strike evasion, and revalidation DoS.
Proposal	Strike when sender is not designated validator	Deter abuse and align with PoC.
Perf	Cache usage in validation path + future invalidation caches	Lower latency and load.